

UAM e-Vote

Système de Vote Électronique Local — UAM

Planning de Développement Collaboratif sur 15 Jours

Binôme : **JOSUE & LATGRAND**

Licence 3 Informatique — Programmation Java Avancé (2026)

Philosophie de Travail : Répartition par "Features"

Afin de respecter les objectifs pédagogiques de l'UFR STA, le travail est réparti par fonctionnalités (modules) plutôt que par couches techniques.

Pourquoi éviter la division Frontend / Backend ?

- **Déséquilibre de compétences** : Un développeur uniquement "Front" ne manipulerait pas JDBC ou le DAO ; un "Back" raterait JavaFX et le CSS.
- **Risque d'évaluation** : Les enseignants interrogent de façon transversale lors de la soutenance.

L'approche par "Feature" (Fonctionnalités)

- Chaque développeur réalise ses modules **de A à Z** (du MCD de sa table à l'interface JavaFX).
- Les deux membres maîtrisent ainsi 100% du programme requis.
- Méthodologie identique aux pratiques d'entreprise agiles.

Technologies & Architecture Cibles

Socle Technique Imposé

- **Langage** : Java 17+
- **Interface Graphique** : JavaFX & FXML, stylisé avec des feuilles de style CSS
- **Base de données** : JDBC avec MySQL / MariaDB
- **Architecture** : Modèle-Vue-Contrôleur (MVC) strict
- **Versionnage** : Git & GitHub (travail collaboratif)

Architecture logicielle (Arborescence)

Le projet adopte une structure rigoureuse pour isoler les responsabilités :

```
src/app/  
├─ model/      (Classes entités : User, Election...)  
├─ dao/        (Accès JDBC : UserDAO, VoteDAO...)  
├─ service/    (Métier et validation)  
├─ controller/ (Gestionnaires d'IHM JavaFX)  
├─ view/       (Fichiers FXML et CSS)  
└─ utils/      (Hachage SHA-256, DBConnection)
```

Répartition Équilibrée des Responsabilités

JOSUE

- **Authentification** (Intégration du hachage SHA-256)
- **Gestion des Utilisateurs** (CRUD & Import CSV)
- **Gestion des Candidats** (CRUD & Programmes)
- **Gestion des Élections** (CRUD & Statuts)

- *Livrables : FXML, Contrôleurs, Modèles, DAO SQL*

LATGRAND

- **Tableau de Bord & Stats** (JavaFX Charts)
- **Vote Électronique** (Parcours complet & Anonymat)
- **Dépouillement & Exports** (Génération PDF/CSV)

- *Livrables : FXML, Contrôleurs, Modèles, DAO SQL, Moteurs d'export*

JOUR 1 : Phase Commune d'Analyse & Conception

Activités initiales clés réalisées ensemble

- ♦ **Analyse du Cahier des Charges** : Identification des règles de scrutin (Délégués, Chefs de dpt, etc.).
- ♦ **MCD / Diagramme de classe / Base de données SQL** : Création de la base et écriture des tables de base (`users`, `elections`, `candidats`, `votes`).
- ♦ **Configuration du workspace** : Initialisation de l'arborescence MVC (`src/app/`) et liaison avec un dépôt distant partagé sur GitHub.
- ♦ **Connexion de base** : Implémentation du fichier `DBConnection` pour centraliser l'accès JDBC.

JOURS 2 & 3 : Authentification & Noyau Utilisateur

JOSUE — Authentification Sécurisée

- Création de l'interface graphique de connexion (Login FXML/CSS).
- Développement de ``PasswordHasher`` (SHA-256) pour le hachage automatique du matricule en tant que mot de passe initial.
- Implémentation de ``authentifier()`` dans ``UserDAO``.
- Gestion de la session utilisateur et déconnexion.

LATGRAND — Gestion des Utilisateurs (CRUD)

- Création de l'entité ``User`` et de ``UserDAO``.
Conception de la vue d'administration des
- comptes (IHM TableView JavaFX).
Création d'utilisateur avec hachage du
- matricule directement dans le DAO.
Formulaires de modification/suppression et
- recherche multicritère.

JOURS 4 & 5 : Gestion des Élections & Candidatures

JOSUE — Gestion des Élections

- Création de l'entité `Election` et de `ElectionDAO`.
- Développement de l'IHM de création de scrutin (Titre, type, dates d'ouverture).
- Gestion des statuts du cycle de vie de l'élection (En préparation, Ouverte,...)
- Implémentation des filtres de scrutins selon le profil de l'électeur.

LATGRAND — Gestion des Candidatures

- Création de l'entité `Candidat` et de `CandidatDAO`.
- Conception de l'interface d'ajout de candidat avec association directe à son élection.
- Enregistrement du profil (Nom, profession, programme).
- Gestion de l'import d'image locale pour la photo d'IHM.

JOURS 6 & 7 : Vote Électronique & Intégration

JOSUE — Importation CSV (Bonus)

- Mise en place du parseur de fichiers CSV pour charger massivement les listes d'électeurs autorisés.
- Gestion et capture des erreurs de format et des doublons de clés primaires (id).

LATGRAND — Processus de Vote

- Création de `Vote` et de `VoteDAO`.
- Développement du parcours utilisateur :
Sélection de l'élection active -> Liste graphique des candidats -> Boîte de confirmation -> Soumission.
- Anonymat garanti au niveau de la structure SQL.

JOURS 8 & 9 : Administration, Visualisation & Sécurité

JOSUE — Sécurité & Unicité

- Mise en œuvre de la contrainte d'unicité SQL
`UNIQUE(election\_id, utilisateur\_id)` pour bloquer définitivement le double vote.
- Blocage immédiat et gel visuel de l'interface utilisateur dès la confirmation du vote.

Restitution visuelle avec les composants
`PieChart` et `BarChart` JavaFX.

LATGRAND — Dashboard & Graphiques

- Création de l'interface globale d'administration avec barre latérale.
- Affichage des KPI en temps réel (taux de participation, inscrits).
- Développement de l'algorithme de dépouillement automatique.
-

JOURS 10 & 11 : Consolidation & Exports des Résultats

JOSUE — Robustesse & Robustesse

- Traitement global des exceptions SQL et JavaFX de l'application.
- Conception de boîtes de dialogue claires et ergonomiques (Alert Dialogs) pour l'utilisateur en cas d'erreur.
- Validation renforcée des données saisies aux formulaires.

LATGRAND — Exports des Résultats

- Moteur d'extraction des résultats au format Excel / CSV.
- Intégration d'une bibliothèque PDF (OpenPDF / iText) pour générer et imprimer le procès-verbal officiel à la clôture de l'élection.

JOURS 12 & 13 : Chasse aux Bugs & Optimisations

Fiabilisation et Stabilisation de l'application

- **Tests de charge** : Simulation de votes massifs avec un jeu de données de test allant jusqu'à 100 électeurs.
- **Optimisations des accès DB** : Nettoyage de la couche DAO et fermeture systématique des connexions JDBC et `PreparedStatement` pour éviter les fuites de mémoire.
- **Polissage UI/UX** : Alignement esthétique des interfaces FXML à l'aide de styles CSS enrichis.

JOUR 13 — Documentation Technique

Génération complète de la JavaDoc pour l'ensemble du projet. Nettoyage du code source, indentation et écriture de

JOURS 14 & 15 : Finalisation des Livrables & Soutenance

JOUR 14 — Production des livrables

- Rédaction du rapport technique officiel de 10 à 20 pages détaillant l'architecture logicielle et le Modèle Physique de Données.
- Création du manuel utilisateur complet et illustré.
- Extraction du script SQL de base de données final incluant les contraintes.
- Préparation du support de présentation (PowerPoint).

JOUR 15 — Validation Final

- Simulations à blanc de l'épreuve orale et scénarisation de la démonstration en direct (Live Demo).
- Contrôle de conformité par rapport au cahier des charges de l'UFR STA.
- Dépôt et livraison finale du projet complet sur GitHub.